

Optymalizacja Dróg Patrolowych

Mikołaj Miszka
Uniwersytet Gdański

10 czerwca 2026

Streszczenie

Niniejszy dokument przedstawia wyniki badań nad zastosowaniem klasycznych heurystyk oraz zaawansowanych algorytmów metaheurystycznych sztucznej inteligencji (AI) w rozwiązaniu Problemu Komiwożera (TSP) w kontekście optymalizacji dróg patrolowych. Zaimplementowano i przetestowano algorytm Najbliższego Sąsiada (NN), Algorytm Genetyczny (GA), Algorytm Mrówkowy (ACO), Symulowane Wyżarzanie (SA) oraz optymalizator LKH-3. Przeprowadzono również analizę porównawczą po zastosowaniu modyfikacji inicjalizacyjnych oraz zaawansowanych operatorów krzyżowania. Badania przeprowadzono na asynchronicznym serwerze FastAPI zintegrowanym z silnikiem OSRM, co pozwoliło na obliczenia na rzeczywistych odległościach drogowych.

Spis treści

1	Wstęp i Definicja Problemu	2
2	Wykorzystane Algorytmy i Modele Matematyczne	2
2.1	Najbliższy Sąsiad (Nearest Neighbor - NN)	2
2.2	Algorytm Genetyczny (PyGAD)	2
2.3	Algorytm Mrówkowy (Ant Colony Optimization - ACO)	3
2.4	Symulowane Wyżarzanie (Simulated Annealing - SA)	3
2.5	LKH-3	4
3	Metodologia Badań	4
4	Wyniki Eksperymentów	5
4.1	Złożoność czasowa	5
4.2	Jakość rozwiązań	5
5	Wnioski i Podsumowanie	7

1 Wstęp i Definicja Problemu

W nowoczesnych systemach logistycznych oraz systemach zarządzania flotą (np. wyznaczanie tras dla radiowozów patrolowych, kurierów), kluczowym wyzwaniem jest minimalizacja kosztów operacyjnych, takich jak czas przejazdu i zużycie paliwa. Problem ten sprowadza się matematycznie do Asymetrycznego Problemu Komiwojażera (ang. *Asymmetric Traveling Salesperson Problem* - ATSP).

Celem systemu jest wyznaczenie najkrótszej, zamkniętej trasy (Cyklu Hamiltona), która odwiedzi zbiór zdefiniowanych N punktów (adnotacji geograficznych) dokładnie jeden raz i powróci do punktu startowego.

2 Wykorzystane Algorytmy i Modele Matematyczne

W ramach środowiska badawczego zaimplementowano pięć zróżnicowanych architektonicznie podejść do rozwiązywania problemu ATSP. Dla każdego z nich przeprowadzono proces strojenia hiperparametrów w celu osiągnięcia balansu pomiędzy czasem obliczeń a dokładnością.

2.1 Najbliższy Sąsiad (Nearest Neighbor - NN)

Zwykła heurystyka zachłanna. Algorytm w każdym kroku wybiera nieodwiedzone miasto, które w danej chwili znajduje się najbliżej obecnego położenia. Metoda ta charakteryzuje się natychmiastowym czasem wykonania, jednak jest wysoce podatna na utykanie w minimach lokalnych, co objawia się przecinającymi się krawędziami na końcu wyznaczanego cyklu. W dalszej części badań posłużyła ona jako solidny fundament inicjalizacyjny dla algorytmów bardziej zaawansowanych.

2.2 Algorytm Genetyczny (PyGAD)

Metaheurystyka inspirowana procesem ewolucji biologicznej. Wykorzystano bibliotekę PyGAD. Pojedynczy osobnik reprezentuje kolejność odwiedzania miast. Funkcja przystosowania (ang. *fitness*) dąży do maksymalizacji odwrotności całkowitego dystansu trasy:

$$f(x) = \frac{1}{\sum_{i=1}^N d(c_i, c_{i+1})} \quad (1)$$

Aby zapobiec przedwczesnej zbieżności, zagwarantować przeszukiwanie przestrzeni bez duplikacji węzłów oraz poprawić jakość wyników, po analizie i wprowadzonych modyfikacjach przyjęto następujące założenia:

- **Inicjalizacja:** Zamiast w pełni losowej populacji, początkowa generacja jest zasilana osobnikiem wygenerowanym przez algorytm Najbliższego Sąsiada (NN). Zapewnia to ewolucji znacznie lepszy punkt startowy.
- Liczba pokoleń (`num_generations`): 400
- Wielkość populacji (`sol_per_pop`): 100
- Liczba rodziców (`num_parents_mating`): 100
- Mutacja: typ *swap*, $P_m = 0.02$

- **Krzyżowanie:** typ *PMX* (*Partially Matched Crossover*) - zmieniono z bazowego krzyżowania jednopunktowego (*single point*), co radykalnie poprawiło przekazywanie poprawnych sub-tras (cech dziedzicznych) bez uszkodzania struktury chromosomu komiwojażera.

2.3 Algorytm Mrówkowy (Ant Colony Optimization - ACO)

Algorytm z dziedziny inteligencji roju, symulujący zachowanie mrówek poszukujących optymalnej ścieżki do pożywienia. Prawdopodobieństwo wyboru kolejnego miasta j przez mrówkę k znajdującą się w mieście i opisuje wzór:

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in N_k} [\tau_{il}]^\alpha [\eta_{il}]^\beta} \quad (2)$$

gdzie τ_{ij} to ilość feromonu na krawędzi, a η_{ij} to heurystyka widoczności. W optymalnej konfiguracji przyjęto następujące parametry:

- Liczba mrówek (`num_ants`): 40
- Liczba iteracji (`num_iterations`): 200
- Waga feromonu (α): 1.25
- Waga heurystyki odległości (β): 5.0
- Współczynnik parowania feromonu (`evaporation_rate`): 0.20
- Stała wzmocnienia śladu (Q): 100.0

2.4 Symulowane Wyżarzanie (Simulated Annealing - SA)

Algorytm ten rozpoczyna działanie od rozwiązania początkowego oraz wysokiej "temperatury", która stopniowo maleje w czasie. Poniżej przedstawiono zmodyfikowany proces krok po kroku[1]:

1. **Inicjalizacja:** Rozpoczęcie algorytmu z początkowym rozwiązaniem S_0 , które jest wyznaczane za pomocą **algorytmu Najbliższego Sąsiada (NN)**, oraz początkową temperaturą T_0 . Temperatura kontroluje prawdopodobieństwo akceptacji gorszych rozwiązań podczas przeszukiwania przestrzeni stanów.
2. **Przeszukiwanie sąsiedztwa:** W każdym kroku generowane jest nowe rozwiązanie S' poprzez wprowadzenie niewielkiej zmiany (perturbacji, np. mutacji *swap*) do obecnego rozwiązania S .
3. **Ocena funkcji celu:** Nowe rozwiązanie S' jest ewaluowane przy użyciu funkcji celu. Jeśli S' zapewnia lepsze rozwiązanie niż S , jest ono akceptowane wprost jako nowe rozwiązanie.
4. **Prawdopodobieństwo akceptacji:** Jeśli S' jest gorsze od S , nadal może zostać zaakceptowane z prawdopodobieństwem zależnym od aktualnej temperatury (T) oraz różnicy wartości funkcji celu (ΔE). Prawdopodobieństwo akceptacji opisuje równanie termodynamiczne:

$$P(\text{akceptacja}) = e^{-\frac{\Delta E}{T}} \quad (3)$$

5. **Harmonogram chłodzenia:** Po każdej iteracji temperatura jest obniżana zgodnie z predefiniowanym harmonogramem chłodzenia, co determinuje, jak szybko algorytm ulega zbieżności. Powszechne harmonogramy obejmują chłodzenie liniowe, wykładnicze lub logarytmiczne.
6. **Warunek stopu:** Algorytm kontynuuje pracę aż system osiągnie bardzo niską temperaturę (brak znaczących ulepszeń) lub osiągnięta zostanie określona liczba iteracji.

W ramach strojenia środowiska dla tego projektu wykorzystano chłodzenie wykładnicze:

- Temperatura początkowa (T_0): 10000.0
- Temperatura końcowa zamarzania: 1.0
- Współczynnik stygnięcia (`cooling_rate`): 0.98
- Liczba prób mutacji na jedną epokę temperatury: 50

2.5 LKH-3

W ramach ustalenia absolutnej granicy odniesienia dla badanych metaheurystyk, zaimplementowano algorytm **Lin-Kernighan-Helsgaun (LKH-3)**. Jest to obecnie jeden z najbardziej zaawansowanych i najskuteczniejszych solverów dla wariacji problemu komiwojażera na świecie. Zgodnie z mechaniką opisaną przez K. Helsgauna [2], środowisko LKH-3 pozwala na rozwiązywanie silnie ograniczonych problemów routingu pojazdów (VRP) oraz asymetrycznych grafów (ATSP) – z jakimi mamy do czynienia pobierając rzeczywiste odległości z sieci drogowej w systemie OSRM. Algorytm ten realizuje to zadanie w dwóch kluczowych krokach:

1. **Transformacja grafu:** LKH-3 w locie sprowadza badany problem asymetryczny (ATSP) do standardowego, symetrycznego problemu komiwojażera (STSP). Osiąga to m.in. poprzez duplikowanie wierzchołków grafu oraz narzucanie wysoce restrykcyjnych funkcji kary (ang. *penalty functions*) na nielogiczne przejścia.
2. **Wymiany k -opt:** W przeciwieństwie do prostych heurystyk zamieniających dwie krawędzie (2-opt), LKH implementuje wielokrawędziowe przeszukiwanie sąsiedztwa (dynamiczne k -opt). System potrafi zidentyfikować i "rozplątać" skomplikowane subcykle na mapie w czasie rzędu ułamków sekundy.

3 Metodologia Badań

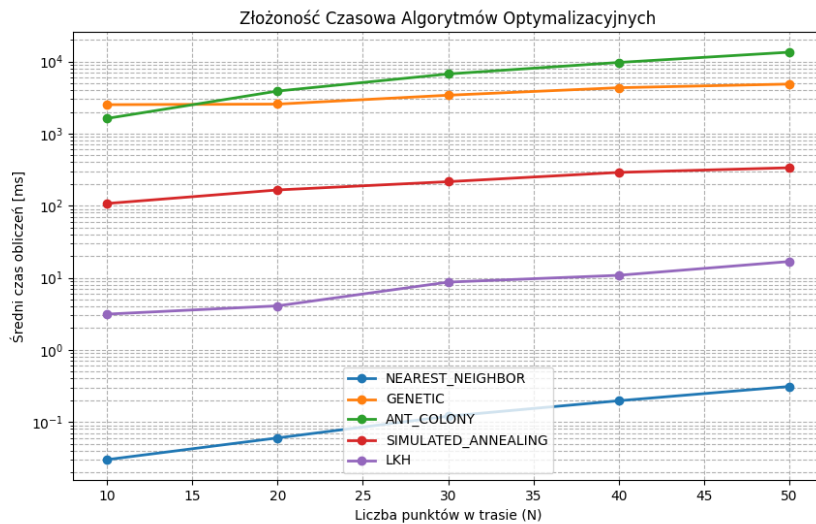
Środowisko testowe uruchomiono w architekturze mikroserwisowej (Docker Compose). Trzon logiki zaimplementowano w środowisku Python (FastAPI). Rzeczywiste odległości drogowe pobierano z silnika **OSRM (Open Source Routing Machine)** za pomocą zoptymalizowanego endpointu `/table/v1/driving/`.

Benchmark zrealizowano za pomocą zautomatyzowanego skryptu w języku Python. Punkty (od $N = 10$ do $N = 50$) były losowane w obrębie Trójmiasta. Każdy algorytm uruchamiano wielokrotnie, a wyniki uśredniano, aby wyeliminować losowość wynikającą z natury algorytmów stochastycznych.

4 Wyniki Eksperymentów

4.1 Złożoność czasowa

Rysunek 1 obrazuje zestawienie średniego czasu obliczeń (w milisekundach) względem rozmiaru problemu. Na osi Y zastosowano skalę logarytmiczną, co pozwala na obiektywne porównanie algorytmów o skrajnie różnej architekturze.

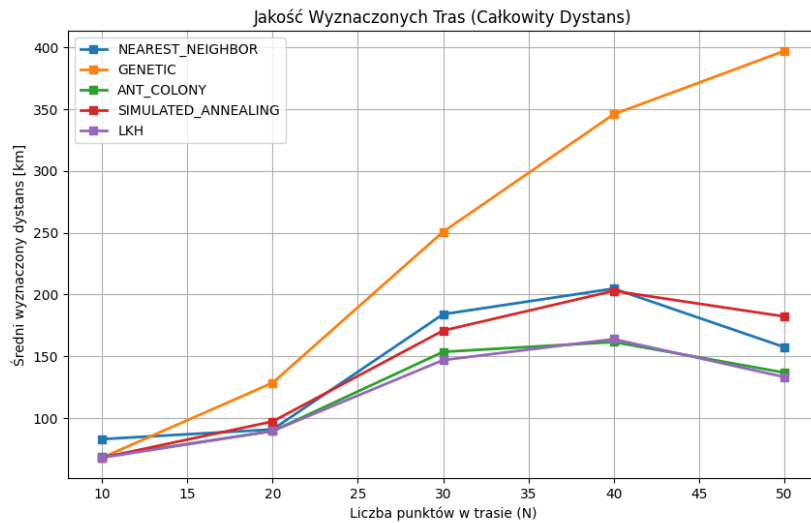


Rysunek 1: Skalowalność czasowa testowanych algorytmów (skala logarytmiczna).

Najbliższy Sąsiad (NN) wykonuje się najszybciej (ułamki milisekundy), co wynika z braku iteracyjnego przeszukiwania globalnego. Warto zauważyć, że wykorzystanie NN do inicjalizacji algorytmów GA i SA nie wpłynęło negatywnie na ich sumaryczny czas wykonania. Skompilowany w języku C optymalizator LKH-3 wykazuje rewelacyjną złożoność czasową, rozwiązując problem dla $N = 50$ w zaledwie kilkanaście milisekund. Symulowane Wyżarzanie (SA) stanowi rozsądny kompromis (ok. 10^2 ms), natomiast implementacje populacyjne (ACO oraz GA) dla problemów wielkości $N \geq 40$ wymagają od kilku do kilkunastu sekund czasu procesora.

4.2 Jakość rozwiązań

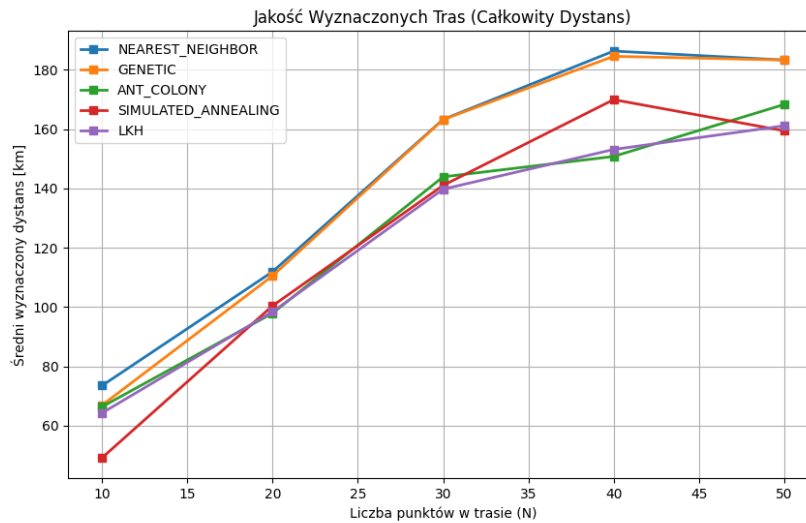
Aby w pełni zobrazować skuteczność wprowadzonych modyfikacji do Algorytmu Genetycznego oraz Symulowanego Wyżarzania, wyniki podzielono na dwa etapy badawcze.



Rysunek 2: Implementacja pierwotna: Algorytm Genetyczny z losową inicjalizacją i krzyżowaniem *single point*; SA z losową inicjalizacją.

Na Rysunku 2 przedstawiono pomiary dla pierwotnych, naiwnych wariantów implementacji. O ile algorytm LKH-3 oraz ACO radziły sobie świetnie, wyznaczając dolną granicę optimum globalnego (stabilizacja wyniku poniżej 150 km dla 50 punktów), o tyle Algorytm Genetyczny (krzywa pomarańczowa) ulegał drastycznej degradacji dla $N \geq 30$. Populacja błyskawicznie traciła różnorodność, a błędnie działający w przypadku ATSP operator krzyżowania jednopunktowego (*single point*) prowadził do przedwczesnej zbieżności i blokowania się algorytmu w lokalnych minimach. Skutkowało to całkowitym dystansem zbliżającym się do absurdalnych 400 km.

Z tego powodu zaimplementowano zaktualizowane podejście: wprowadzono zaawansowany operator **PMX (Partially Matched Crossover)**, który chroni prawidłowe połączenia węzłów u potomków, a ponadto algorytmy GA oraz SA otrzymały "wskazówkę" na start w postaci wstępnego rozwiązania wygenerowanego przez **Najbliższego Sąsiada (NN)**.



Rysunek 3: Implementacja zoptymalizowana: Inicjalizacja z wykorzystaniem NN dla algorytmów GA oraz SA, wdrożenie operatora krzyżowania PMX dla GA.

Rysunek 3 prezentuje drastyczną zmianę zachowania modeli po modyfikacjach. Symulowane Wyżarzanie (SA - krzywa czerwona) zauważalnie obniżyło końcowy dystans względem pierwotnej wersji, ponieważ stygnięcie następowało wokół dużo lepszego lokalnie punktu startowego.

Fenomenalną poprawę odnotował z kolei Algorytm Genetyczny. Rozwiązanie to niemal całkowicie zniwelowało problem przedwczesnej degradacji – dla dystrybucji $N = 50$ ostateczny dystans spadł z około 400 km do zaledwie ~ 180 km, lądując na podobnym poziomie użyteczności co wynik z samego algorytmu NN, dając szansę na dalsze obniżanie wyników przy dołożeniu większej liczby epok.

5 Wnioski i Podsumowanie

Przeprowadzone badania empiryczne pozwoliły na pełną ocenę użyteczności algorytmów optymalizacyjnych w systemach logistycznych bazujących na rzeczywistych danych geograficznych (ATSP).

1. **Perfekcja LKH-3:** Zastosowanie wielokrawędziowych wymian (k -opt) skompilowanych w środowisku C całkowicie deklasuje implementacje pythonowe. LKH-3 dostarcza optymalne trasy w ułamku sekundy, stanowiąc docelowe rozwiązanie dla systemów produkcyjnych.
2. **Jakość vs. Czas w Algorytmie Mrówkowym (ACO):** Algorytm ten udowodnił swoją wybitną skuteczność, osiągając wyniki dystansowe identyczne z referencyjnym LKH-3 bez wsparcia z zewnątrz. Jest to jednak okupione bardzo wysokim kosztem czasowym.
3. **Znaczenie odpowiednich operatorów ewolucyjnych (GA):** Pierwotne testy ukazały całkowitą nieprzydatność klasycznego krzyżowania *single point* w problemach permutacji (TSP/ATSP). Zmiana operatora na **PMX** połączona z inteligentną inicjalizacją (NN) przywróciła użyteczność Algorytmu Genetycznego, zmniejszając

szając długość wyliczanych tras o ponad połowę. Potwierdza to fakt, że algorytmy populacyjne wymagają precyzyjnego dostrojenia do natury problemu.

4. **Synergia heurystyk (NN + Metaheurystyki):** Naiwny algorytm Najbliższego Sąsiada sam w sobie jest nieprzewidywalny i często daje suboptimalne wyniki, jednak wykorzystany jako inicjalizator dla Symulowanego Wyżarzania (SA) oraz Algorytmu Genetycznego (GA) wykazuje doskonałą synergię, za darmo poprawiając końcową zbieżność bardziej zaawansowanych metod bez strat czasowych.

Literatura

- [1] GeeksforGeeks. What is simulated annealing?
- [2] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde University*, 12:966–980, 2017.